

SHUNYA

Gaming Engine

Technical Proposal & Engagement Plan



Prepared by:	ChicMic Studios
Prepared for:	Shunya Platform
Date:	16th April 2026
Version:	3.0 — CONFIDENTIAL
Engagement:	Fixed-Price · Phased Delivery

01 • Executive Summary

ChicMic Studios has conducted a thorough review of the Shunya Gaming Engine Scope Document and Non-Functional Requirements document provided. This proposal presents our technical engagement plan for designing and delivering Shunya — a high-precision, skill-based elimination gaming platform engineered to funnel millions of participants through five progressively intense online stages, culminating in an offline, in-person finale that crowns a single ultimate champion.

Shunya is not a casual gaming application. It is a precision-engineered competitive infrastructure — combining adaptive cognitive assessment, server-side validated game logic, anti-cheat architecture, and a scalable backend capable of supporting 1M+ monthly active users at peak concurrency. Our proposal reflects this ambition with realistic timelines, phased delivery, and rigorous engineering standards.

ENGAGEMENT TYPE	TOTAL TIMELINE
Fixed-Price • Phased Delivery	~8 Weeks

Games Engine & Games	Admin Panel	DevOps & Infra	Load Testing	Excluded
				- Unit Testing
5 Stages • Games • Scoring Engine • WebView Integration	Anti-Cheat, Admin Panel, Question Bank CMS, 4 Environments			Due to Timeline Constraints

Note: Items marked in yellow are currently out of scope and will be handled by the Source Fuse team. We can provide support if required.

02 • Project Understanding

2.1 What Shunya Is

Shunya is a mass-participation, skill-based elimination competition platform. Beginning with potentially millions of registered players, it applies a progressive filtration architecture across five distinct online stages — each testing a deeper cognitive dimension — before a physical finale determines a single champion.

2.2 The Funnel Architecture

The platform operates across three distinct phases:

- **Pre-Bucketing (Stages 1–3):** Players are segmented by registration week into cohorts (Buckets). Each bucket independently completes three stages with ~50% elimination per stage. A cohort registering 1,000,000 players produces ~125,000 survivors.
- **Post-Bucketing (Stages 4–5):** All surviving cohorts are merged into a unified competitive pool for two advanced stages testing persistence, strategic depth, and speed under pressure.
- **Offline Finale:** The top few hundred survivors are invited to a physical venue for the final crowning event.

2.3 Game Portfolio

Shunya deploys distinct game types across 5 stages, each engineered to isolate a specific cognitive dimension:

Stage	Theme	Games	Primary Dimension
Stage 1	The Divine Echo of Precision	Object Placement Memory, Sequence Recall, Spot the Difference, GK MCQ	Memory & Knowledge
Stage 2	The Kinetic Pulse	Sliding Puzzle, Colour-Based Sorting, Block Fill, Mathematics MCQ	Reaction Time & Speed
Stage 3	The Mind Labyrinth	Logical Reasoning MCQ, Maze Navigation, Memory Card Matching, Mythology MCQ	Correctness + Speed (Dual)
Stage 4	The Crucible of Will	MCQ Rapid Fire, Skill Memory Test (3×3), Mini Sudoku (6×6), Path Optimisation	Persistence & Strategy
Stage 5	The Final Reckoning	Connections, Advanced Sliding Puzzle, Hard Maze Navigation, MCQ Rapid Fire (Hard)	Mastery Synthesis

2.4 The NFR Mandate

The provided Non-Functional Requirements establish demanding operational baselines that shape every architectural decision in this proposal:

- 100,000 concurrent users during active gameplay bursts
- 10M+ monthly active users (horizontal scaling required)
- Deterministic game state: identical outputs for the same seed and difficulty
- Server-side score validation — client is never trusted
- Android and iOS WebView compatibility (current + N-1 OS versions)
- 80% minimum unit test coverage on all game logic **<not included in phase1>**
- 4 separate environments: Dev, QA, Staging, Production
- Full documentation: event definitions, metadata schema, integration instructions

2.5 User Journey

The following is the end-to-end player experience, traced from first app install through to the offline finale. Each step identifies which system is responsible and what technical event occurs.

Note: We have shared the proposal for the gaming engine only, assuming that the base application will be designed and developed by the client.

Phase A — Registration & Bucket Assignment (Base Platform)

#	Actor	Action	Technical Detail
1	User	Downloads & installs app	Native iOS/Android app from App Store / Play Store
2	User	Registers — name, phone/email, KYC	Base Platform handles registration; creates user_id
3	Base Platform	Assigns user to weekly bucket	bucket_id = Week N based on registration_timestamp; cohort capped after Week 5/6
4	Base Platform	Stores bucket mapping	user_id → bucket_id persisted in Base Platform DB
5	Base Platform	Notifies user of Stage 1 availability	Push notification / in-app alert when Stage 1 window opens for their bucket

Phase B — Game Session Launch (Handoff Point)

This is the critical integration boundary. The Base Platform hands control to the Gaming Engine here by opening a WebView with a signed session token.

#	Actor	Action	Technical Detail
6	User	Taps 'Play Stage N, Game X' in app	Base Platform UI — game selection screen
7	Base Platform	Validates user eligibility	Checks: correct stage, correct bucket window, not already played this session, not eliminated
8	Base Platform	Issues signed session token	JWT signed with shared secret: { user_id, bucket_id, stage_id, game_id, session_id, issued_at, expires_at, hmac_signature }
9	Base Platform	Opens WebView with token	WebView URL: https://games.shunya.gg/{game_id}?session={JWT_TOKEN}
10	Gaming Engine	Receives WebView load request	Game WebApp loads in WebView; parses session token from URL params
11	Gaming Engine	Validates token with backend	Game WebApp calls Gaming Engine Backend: POST /session/init with JWT — backend verifies signature, checks expiry, confirms no duplicate session
12	Gaming Engine Backend	Generates game state	Server creates game instance: generates server_seed, derives initial game state via PRNG(server_seed), stores in Redis with session_id key
13	Gaming Engine Backend	Returns game configuration	Response to WebApp: { game_config, initial_state, session_token_confirmed } — seed is NEVER sent to client
14	User	Game Rules Screen	User will view static game rules that is required for user to play the game
15	User	Game begins	Timer starts server-side at confirmed start event from client

Phase C — Active Gameplay (Gaming Engine Owned)

#	Actor	Action	Technical Detail
16	User	Plays game — makes moves/answers	All interactions captured client-side with timestamps
17	Game WebApp	Emits move events (optional, for validation games)	WebSocket or HTTP events: { session_id, move_type, payload, client_timestamp } — used for anti-cheat and optional move tracking

18	Gaming Engine Backend	Records move log (if applicable)	Stored in Redis for session duration; persisted to DB on session end for audit
19	Gaming Engine Backend	Monitors anti-cheat signals	Checks: input timing patterns (flags sub-100ms consistent sequences), session token validity, replay detection
20	User	Completes game (win/lose/timeout)	Client sends game_complete event OR server-side timer fires
21	Game WebApp	Submits final answer/state	POST /session/submit: { session_id, final_state, client_score_claim } — client score is for UX only, never trusted

Phase D — Score Validation & Webhook Submission (Critical Path)

This is where the Gaming Engine's server-side validation runs and the validated score is transmitted to the Base Platform. The Base Platform is the authoritative consumer of this result.

#	Actor	Action	Technical Detail
22	Gaming Engine Backend	Reconstructs game state server-side	Replays PRNG(server_seed + user_input_hash) to reproduce the exact game state presented to the user — no client data trusted for this
23	Gaming Engine Backend	Validates submitted answers/moves	Compares submitted final_state against server-reconstructed expected_state; calculates authoritative score using scoring formula from scope doc
24	Gaming Engine Backend	Runs anti-cheat final check	Reviews move log timing, checks for session anomalies, validates time-window compliance
25	Gaming Engine Backend	Flags or clears session	CLEARED: proceeds to score emission. FLAGGED: score recorded as 0 or quarantined pending review
26	Gaming Engine Backend	Emits score to Base Platform webhook	POST {base_platform_webhook_url}/game-result with signed payload (see webhook schema below)
27	Base Platform	Acknowledges receipt (HTTP 200)	Base Platform must respond within 5 seconds; Gaming Engine will retry up to 3 times on failure with exponential backoff
28	Game WebApp	Shows result screen to user	Displays score, breakdown, and 'returning to platform' message
29	Game WebApp	Closes WebView / notifies host app	postMessage or deep-link callback to Base Platform app: { session_id, status: 'complete' }
30	Base Platform	Receives WebView close signal	Re-activates native app UI; begins polling or listens for leaderboard update

Phase E — Leaderboard Processing & Elimination (Base Platform)

#	Actor	Action	Technical Detail
31	Base Platform	Aggregates scores from all game results	Collects all validated scores for the stage window; builds cumulative Shunya Points leaderboard per bucket
32	Base Platform	Runs elimination logic	After stage window closes: ranks users by cumulative score; eliminates bottom ~50% (configurable by operator)
33	Base Platform	Updates user status	Qualified users: status = STAGE_N+1_ELIGIBLE. Eliminated users: status = ELIMINATED
34	Base Platform	Notifies users	Push notification: 'You qualified for Stage N+1!' or 'Your journey ends here — thank you for competing'
35	Base Platform	Unlocks next stage for qualified users	UI shows Stage N+1 'Play Now' button for eligible users; locked state for others
36	User	Repeats Phase B–E for each stage	User progresses through Stages 1–5 via the same WebView-launch flow

Phase F — Post-Bucketing Merge & Stages 4–5

#	Actor	Action	Technical Detail
37	Base Platform	Merges all surviving buckets after Stage 3	All Week 1–6 survivors combined into a single unified competitive pool
38	Base Platform	Reconfigures session tokens for Stage 4	JWT payload updated: bucket_id no longer relevant; stage_id = 4, pool = UNIFIED
39	User	Plays Stages 4 and 5 via same WebView flow	Integration pattern identical to Stages 1–3; no changes required from Gaming Engine side
40	Base Platform	Final elimination after Stage 5	Top few hundred players receive Offline Finale invitation

2.6 RNG Architecture - Server Seed + User Input Model

Shunya's game fairness and anti-cheat integrity depend entirely on a deterministic, verifiable, server-authoritative random number generation system. The architecture below implements a dual-seed model: a server-generated secret seed (which the client never sees) combined with a user input hash (making each session uniquely tied to the user's actions). This design satisfies the NFR mandate for determinism while preventing pre-calculation attacks (This is according to the initial understanding and will inform you in advance if any changes are required).

03 · Objectives & Success Criteria

3.1 Delivery Objectives

- Deliver all games as standalone WebApps, each with its own backend, integrated via OAuth + webhook contracts
- Implement the server-side scoring and validation engine that is the sole source of truth for all player scores
- Build an adaptive MCQ engine serving General Knowledge, Mathematics, Logical Reasoning, and Mythology question banks with dynamic difficulty adjustment
- Deploy a robust anti-cheat layer: session token invalidation, input pattern detection, server-side puzzle validation
- Achieve 100K CCU support validated by load testing before production launch
- Maintain 4 separate environments throughout development lifecycle

3.2 Success Criteria

Metric	Target	Validation Method
Concurrent User Support	100,000 CCU	Load Testing Tool
Monthly Active User Capacity	10M+ MAU (horizontal scale)	Architecture review + auto-scale test
Game Load Time (WebView)	Acceptable time	Lighthouse + real device testing
Score Determinism	100% — same seed → same result	Automated determinism test suite
Anti-Cheat Detection	Sub-100ms input flagged	Automated bot simulation test
Environment Parity	4 envs (Dev/QA/Staging/Prod)	Infrastructure audit
OAuth Integration	Zero token leaks, session integrity	Penetration test + session audit

04 · Proposed Solution Overview

Shunya's gaming engine is not a monolithic game — it is a distributed game orchestration platform. Each of the games is a standalone microservice-WebApp with its own backend, communicating with a central Contests Service through defined OAuth and webhook contracts. This architecture ensures independence, scalability, and maintainability at every layer.

4.1 Architecture Pillars

Game WebApps (Frontend)

All games are built as responsive WebApps rendered inside the host app's WebView. Each game is a self-contained unit with its own UI, game logic, and scoring logic. Games communicate with their own backends and with the Contests Service backend via defined events.

Game Backends (Per-Game Microservices)

Each game has a dedicated backend responsible for server-side state initialization (using deterministic seeds), validation of player submissions, score calculation, and event emission to the Contests Service. This ensures the client is never trusted for scoring.

Contests Service Integration

The Contests Service is the client's own backend. Our games integrate with it via: (1) OAuth-based session initialization at game start, and (2) Webhook-based score submission at game end. We will adhere strictly to the integration contract defined by the client's team.

Adaptive MCQ Engine

A shared, stateful MCQ engine serves question banks across Stages 1–5. The adaptive difficulty algorithm (2 consecutive correct → level up; 2 consecutive incorrect → level down) is implemented server-side and is deterministic per session seed. Question banks are rotated per cohort-session to prevent farming and Question banks need to be uploaded via Admin Panel with suggested format.

Anti-Cheat Architecture

Server-side validation for all solvable puzzles (mazes, Sudoku, sliding puzzles, Block Fill) with pre-calculated optimal solution sets. Session token invalidation on detection of abnormal input patterns (consistent responses across all events). Reaction time scoring implements a 0–5 second floor to eliminate pre-click exploits.

4.2 Development Phases

4.2.1 Phase 1 Scope: All Games + Core Infrastructure + Admin Panel + Design

Stage 1 — The Divine Echo of Precision

- Object Placement Memory ($n \times m$ grid, configurable display timer, drag-and-drop reconstruction)
- Sequence Recall (Simon-style, 4-cell grid, audio feedback, growing sequence, single-mistake termination)
- Spot the Difference (side-by-side images, predefined difference regions, speed + accuracy scoring, hint system)
- MCQ — General Knowledge (2-min window, adaptive difficulty engine, image-based questions supported)

Stage 2 — The Kinetic Pulse

- Sliding Puzzle (3×3 , solvability-guaranteed scramble via parity check, abstract image bank, timer scoring)
- Colour-Based Sorting (Ball Sort mechanic, tube stack structure, undo/reset with penalty, completion detection)
- Block Fill (8×8 grid, Flow Free mechanic, multi-color path drawing, server-validated solutions)
- MCQ — Mathematics (2-min window, adaptive difficulty, arithmetic through number theory, graphical questions)

Stage 3 — The Mind Labyrinth

- MCQ — Logical Reasoning (pattern series, analogies, syllogisms, coding-decoding, 2-min adaptive)
- Maze Navigation (procedurally generated, swipe controls, single-optimal-path guarantee, time-based scoring)
- Memory Card Matching (4×4 grid, 3-sec reveal flash, matched-pair lock mechanic, time-bonus scoring)
- MCQ — Mythology (Hindu, Greek, Norse, Egyptian folklore, illustrative artwork in questions, 2-min adaptive)

Stage 4 — The Crucible of Will

- MCQ Rapid Fire (2-strike elimination, 5-min window, mixed themes, persistent strike counter UI)
- Skill Memory Test — Sequence 3×3 (N-Back paradigm, growing sequence levels, single-mistake termination)
- Mini Sudoku Easy — 6×6 (pre-validated unique-solution puzzles, pencil-marking helper, 5-min window)
- Path Optimisation (5–10 checkpoint nodes, TSP variant, efficiency-bonus scoring, pre-calculated optimal path)

Stage 5 — The Final Reckoning

- Connections (NYT-style category grouping, infinite tile regeneration, streak bonuses, 2-min sessions)
- Advanced Sliding Puzzle (3×3 without numbered tiles, larger grid, shorter reference flash)
- Hard Maze Navigation (larger grid, more dead ends, 5-min window)
- MCQ Rapid Fire — Hard (harder question bank, cross-theme questions, 2-strike elimination)

Shared Infrastructure (Phase 1)

- **Adaptive MCQ Engine (server-side, deterministic, question banks, per-cohort rotation)**
- Server-side scoring and validation engine
- OAuth session initialization + webhook score submission integration
- Pre-game static rule screen
- Core anti-cheat layer (input pattern detection, session invalidation, reaction time floor)
- WebView compatibility (Android + iOS, current + N-1)
- Dev and QA environments

4.2.2 Infra + AWS: All Games + Core Infrastructure

1. Automated Infrastructure (IaC & GitOps)

- **Terraform:** We use Terraform to maintain four strictly isolated environments (**Dev, QA, Staging, Prod**). This ensures that infrastructure changes are tested before reaching production.
- **ArgoCD:** By adopting a **GitOps** model, the EKS cluster state is automatically synchronized with our Git repository. This provides a version-controlled audit trail for all deployments.

2. Scaling & Resilience

- **100K CCU & Auto-Scaling:** We are achieving this through **EKS + Karpenter**, which provides rapid, just-in-time node provisioning and "bin-packing" to handle traffic bursts more efficiently and cost-effectively than Fargate.
- **CDN & Image Optimization:** We integrate **CloudFront** with an automated **S3/Lambda@Edge** pipeline for real-time **WebP conversion** and mobile-optimized delivery to ensure low latency and responsive gameplay.
- **Performance Profiling:** Continuous monitoring is handled via **Prometheus and Grafana** to optimize CPU, GPU, and memory utilization, ensuring we meet strict NFR targets for smooth, deterministic performance.
- **Reliability & Security:** We implement **RDS Proxy** to manage PostgreSQL connection spikes and **AWS Secrets Manager** for secure, version-controlled credential handling across all 13 microservices

3. Security & Containerization

- **Secrets Manager:** All sensitive data, including database credentials and OAuth keys, are stored in **AWS Secrets Manager** and injected into pods via the CSI Driver to avoid plain-text environment variables.
- **Optimized Dockerization:** We maintain **13 microservices** using a standardized, multi-stage base image to reduce bloat and meet NFR targets for low CPU and memory utilization.

4. Performance & Monitoring

- **Static Frontend:** The game Webapp is served via **S3 and CloudFront**, utilizing **WebP conversion** and lazy-loading to minimize mobile latency.
- **Observability:** Centralized **Prometheus and Grafana** dashboards provide real-time profiling of CPU, GPU, and memory usage.

4.2.3 Phase 2 Future Scope: All Games + Core Infrastructure

- Unit testing with 80% test coverage will be delivered in Phase 2. Efforts for unit testing and coverage have not been included in the current estimate **{not in current scope}**.
- Any change requests mentioned above that were not part of the initial discussion will be considered for Phase 2.
- We have considered a basic admin panel that will provide options to upload new questions, replace existing ones, and add entries to the question bank. No other information—such as images, game stats, or related content—will be editable from the admin side. Any advanced features, if required, should be communicated during Phase 1. These will be implemented in a subsequent phase with an additional estimate and cost, depending on the specific feature requirements.
- We will initiate load testing and aim to cover all games within a 7–8 week milestone.
- Reports included within the current scope will be initiated during the development phase. However, due to the dependency on multiple parameters, there is a possibility that their final delivery may extend beyond Phase 1 and be completed in Phase 2.
- Additional anti-cheat enhancements and improvements will be implemented in Phase 2.
- Complete documentation for the projects will be provided in phase 2 **{included in current scope but will be delivered later}**.

Note:

All paid tools and third-party services will be provisioned by the client, or any associated costs will be borne by the client.

Due to timeline constraints, the following reports are not included within the scope of Phase 1:

- Security validation report
- Unit test coverage report

05 · Technical Architecture

5.1 Recommended Technology Stack

Layer	Technology	Justification
Game Frontend (WebApp)	React/Gaming Engine + Javascript/Typescript + Vite	WebView-compatible, fast HMR, mobile-responsive, tree-shakeable bundles for performance
Game Animations	CSS Animations + GSAP (lightweight)	GPU-accelerated; works within WebView constraints
Game Backend (Per-game)	Node.js (NestJS) + TypeScript	Consistent with modern API patterns; modular; strong typing for deterministic logic
Adaptive MCQ Engine	Node.js service + Redis (session state)	Redis ensures low-latency stateful session tracking across distributed instances
Score Validation + Anti-Cheat	Server-side NestJS module	Centralized, auditable, impossible to spoof from the client
Database	PostgreSQL (primary) + Redis (cache/sessions)	ACID compliance for score integrity; Redis for sub-millisecond session reads
Question Bank Storage	PostgreSQL + Admin CMS	Structured querying with operator configurability
Message Queue (async events)	AWS SQS / Bull Queue	Decoupled webhook delivery to Contests Service; retry logic on failure
Infrastructure	AWS (EKS + ALB + RDS + ElastiCache)	Horizontal scaling; managed services reduce ops overhead
CDN + Image Delivery	AWS CloudFront + S3	Optimized image delivery for mobile WebView; global edge caching
CI/CD	Code Pipeline + Docker	Automated test → build → deploy pipeline per environment
Load Testing	K6/JMeter	
Monitoring	AWS CloudWatch	Real-time performance, error tracking, CCU dashboards

5.2 Determinism & Consistency Implementation

The NFR mandates that for a given seed, all game state and scoring must be identical. Our implementation approach:

- All randomization (maze generation, puzzle scrambling, MCQ question selection, object placement) uses a seeded pseudo-random number generator (PRNG) — specifically seedrandom.js or an equivalent deterministic library.
- Seeds are generated server-side at session initialization and stored in Redis. The seed is never exposed to the client directly.
- Game state reconstruction (for validation) replays the PRNG with the same seed to verify submitted answers against the server-calculated expected state.
- MCQ question sets are locked per cohort-session at the session initialization phase and served from a snapshot, preventing mid-session question bank updates from affecting ongoing games.

5.3 Scalability Architecture

Supporting 100K concurrent users and 10M+ MAU requires horizontal scaling at every layer:

- Stateless Game WebApps served via CloudFront CDN — zero backend dependency for UI rendering
- Game Backends deployed as auto-scaling ECS Fargate services behind an Application Load Balancer
- Redis Cluster (ElastiCache) for session state — eliminates single-node bottlenecks
- PostgreSQL with read replicas for score ledger reads; primary used only for writes
- Asynchronous webhook delivery via SQS — decouples score submission from real-time game flow
- Pre-warmed game instances during cohort competition windows to eliminate cold-start latency

— STUDIOS —

06 · Development Approach, Methodology & Payment Structure

6.1 Delivery Methodology

ChicMic Studios will follow a sprint-based Agile delivery model with 2–3 week sprints, bi-weekly client demos, and a strict definition of done that includes completed features and a manual QA testing report.

6.2 Cost Estimation

6.2.1: Game Engine and Games

Stage	Features/Games	Comment
0	Project Setup: initialize projects, git and architecture designs Documentation Changelog Generation	Note: 1. Once configured the mcq can be used in multiple stages just by initializing using the different predefined configs so we have excluded estimate for the design also as assuming similar component will be used 2. We have assumed that admin need to upload images along with the data and images need to map with the questions bank and we will share you the sample for the same
1	Object Placement Memory (4×1)	
	Sequence Recall	
	Spot the Difference	
	MCQ — General Knowledge	
2	Sliding Puzzle (3×3 with numbered tiles)	
	Colour-Based Sorting (Ball Sort)	
	Block Fill — Flow / Dot Connect (8×8)	
	MCQ — Mathematics	
3	MCQ — Logical Reasoning	
	Maze Navigation	
	Memory Card Matching (4×4)	
	MCQ — Mythology	
4	MCQ Rapid Fire (2 incorrect = out)	
	Skill Memory Test — Sequence (3×3)	
	Mini Sudoku (6×6, Easy)	
	Path Optimisation	
5	Connections (3×3 tile grouping)	
	Sliding Puzzle Advanced (3×3, no numbered tiles)	
	Maze Navigation (Hard — fog of	

	war)	
	MCQ Rapid Fire — Hard (2 incorrect = out)	
6	SHARED INFRASTRUCTURE: - Shared Game Infra Structure - Question Bank Management - Game Engine	

6.2.2: Admin Panel

Module	Task	Comment
Auth	Login	
Dashboard	Global stats and graphs	we will be providing an minimal dashboard for the admin panel
Game management	Stats	1. We are considering a system to manage game configuration JSON data, such as adding new levels, using only the tools specific to the game.
	Level creation mode for specific games according to the game requirement and management	2. We have not considered a CMS-like structure; for example, users will not be able to edit assets through the admin panel for all games except spot the difference and sliding puzzle where you need to supply the images.
	Level, difficulty and input configs	3. Users will only be able to create specific types of JSON data through the interface, and only the data required for creating those types will be within scope. 4. Per game configs will be the following: a. Object Placement Memory: Grid size, and display time, time allotted b. Sequence Recall: Grid size, initial sequence size, time per flash, time allotted c. Spot the Difference: Image pairs (along with differences in them), base points, penalty for hints, speed bonus, number of hints, time allotted
Anti cheat module	Configurations	d. MCQ: Question bank, Whether rapid fire or normal, time allotted, whether negative marking is allowed, Score awarded per question
	Flagged Users	e. Sliding Puzzle: grid size, Image, Time allotted, Score awarded, anti exploit timer f. Colour-Based Sorting: Efficiency bonus, speed bonus, base points, time allotted, number of tubes
	Tap detection, highlight correct finds, penalty UI	g. Block Fill: Grid size, time allotted, h. Maze: Time Allotted, score allotted

		i. Memory Card Matching: Grid size, base points, reaction time bonus, time allotted j. Memory Test: Grid size, score allotted, Maximum sequence length k. Sudoku: Grid size, time allotted, number of prewritten numbers, score allotted l. Path Optimization: Grid size, whether to include weighted edges, whether to add blocked edges, score allotted m. Connections: Grid size, Words, time allotted, score allotted
--	--	--

6.2.3: DevOps:

Service	Component	Note
Infrastructure as Code	Terraform	1. We have considered all games as independent components, with no dependencies on each other. Accordingly, each game will be designed and deployed as a separate microservice. This approach requires setting up multiple CI/CD pipelines and corresponding infrastructure configurations. Based on this assumption, we have outlined the necessary tasks for infrastructure management. Additional details have also been provided in the proposal. 2. We can achieve these same objectives using AWS ECS with minimal architectural adjustments. While ECS can be more cost-effective and simpler to manage for smaller workloads, it may lack the granular scaling efficiency and ecosystem support that Kubernetes (EKS) provides for a massive scale of 100K concurrent users and 10 million monthly active users.
Code Quality Checks		
Continuous Delivery (GitOps)	ArgoCD	
Compute Scaling	EKS + Karpenter	
Database	RDS PostgreSQL	
Static Frontend	S3 + CloudFront	
CI Pipeline	AWS CodePipeline	
Secrets & Security	Secrets Manager	
Container Registry	Amazon ECR	
Containerization	Docker	
Monitoring	Alerts	
	grafan prometheus	
Security Audit		

6.2.4: Load Testing:

Category	Task
Setup	k6 tool installation, reporter config, CI/CD pipeline integration
	Test environment config — Dev env endpoints, auth tokens, test data seeding
	Grafana dashboard setup for load test metrics (k6 + Prometheus integration)
Script Writing	k6 script — MCQ session (WebView init → JWT → questions → score submit)
	k6 script — WebView session init + JWT validation + score webhook
	k6 script — Redis session state stress (100K concurrent reads/writes)
	k6 script — PostgreSQL read/write load (score submission + leaderboard)
	k6 script — SQS webhook queue stress (concurrent score submissions)
	k6 script — WebSocket load (move events — Maze, Sequence, Skill Memory)
	k6 script — CloudFront CDN throughput (100K concurrent WebView loads)
	Anti-cheat timing simulation — bot pattern injection, sub-100ms detection
Smoke Test	Baseline smoke test — all scripts on Dev env at 100 CCU, verify execution
	Fix script issues found in smoke test, re-run and validate
Ramp Testing	Staging ramp test — 10K CCU, baseline P50/P95/P99 latency per service
	Staging ramp test — 50K CCU, identify first bottlenecks
	Staging full stress — 100K CCU peak load, measure degradation points
	Spike test — burst from 1K to 100K CCU, measure Karpenter response time
	Soak test — 50K CCU sustained 30 mins, detect memory leaks / connection drain
	WebSocket load test — 100K concurrent move event streams under full load
	SQS backpressure test — 100K concurrent score submissions, queue depth check
Analysis	Results analysis — P50/P95/P99 per service, error rates, CPU/memory (Grafana)
	Bottleneck identification report — rank issues by severity and service
Tuning	Karpenter auto-scaling tuning — node provisioning thresholds, bin-packing
	RDS connection pooling tuning — RDS Proxy config, max connections under 100K
	Redis ElastiCache tuning — eviction policy, shard sizing, keyspace hit rate
	CloudFront cache policy tuning — TTLs and cache behaviour under load
	BE bottleneck fixes — N+1 queries, DB indexes, session read optimizations
	SQS retry/backoff config tuning based on backpressure test results
Validation	Re-run 100K CCU load test post-tuning — confirm all bottlenecks resolved
	Production certification test — 100K CCU on Prod env, final sign-off
Reporting	Load test report — P99 latencies, CCU achieved, scaling behaviour, sign-off doc

6.3 Payment Structure and Release

Upon approval of this proposal, we will share a detailed milestone plan outlining the deliverables, timelines, and cost for each milestone. For every milestone, 50% payment will be required upfront before initiation, and the remaining 50% will be due upon completion of that milestone.

Note: The release will only be held in case of major P0 or P1 bugs that block production deployment. Any lower-priority issues will be addressed in the subsequent milestone.

6.4 Resources

Resources	Responsibility
Project Manager	Owns end-to-end delivery, sprint planning, and client communication. Manages timelines, risk register, change requests, and milestone sign-offs across all phases
Game Developer	Builds all game WebApps for Android/iOS WebView. Implements game mechanics, animations, move event emission etc
Backend Developer	Builds per-game microservices, server-side scoring engine, RNG architecture (server seed + PRNG), adaptive MCQ engine , anti-cheat layer, and OAuth/webhook integration with the Base Platform.
DevOps & Infra Engineer	Provisions and manages all 4 environments on AWS (ECS, RDS, Redis, CloudFront). Owns CI/CD pipelines, monitoring, and load testing for 100K CCU validation.
Game Designer	Designs all game UIs and pre-game animations for mobile WebView. Maintains a consistent visual language across games and produces culturally neutral, device-responsive assets.
QA (Manual Testing)	Covers functional, integration, device compatibility, determinism, anti-cheat simulation, and network resilience testing across all games and environments.

7 • Communication & Governance

Reporting Cadence

- Bi-weekly sprint demos with live game previews on the QA environment
- Weekly async progress report (Slack) covering: completed items, upcoming sprint goals, blockers

Change Request Process

Any change to the agreed game specifications, scoring logic, or integration contract outside the scoped configuration parameters will be handled via a formal Change Request (CR) process:

- CR raised by either party with written description of change and business justification
- ChicMic Studios provides impact assessment (timeline + cost) within 3 business days
- Change implemented only upon written approval; sprint plan updated accordingly

